파이썬으로 배우는 동적계획법

Dynamic Programming — 피보나치 수열로 재귀와 비교하기

CodeWithCathy

동적계획법(DP)이란?

큰 문제를 작은 문제로 쪼개고, 한 번 푼 답은 저장해서 다시 쓰는 방법

Dynamic Programming, 줄여서 DP라고 불러요. 이미 계산한 결과를 기억해 두기 때문에 같은 계산을 반복하지 않습니다.

DP의 핵심 한 단어

기억하기 (Memo)

DP를 쓸 수 있는 두 가지 조건

1

중복되는 부분 문제

같은 작은 문제가 여러 번 반복해서 나온다

예) fib(5)를 구하는 중에 fib(3)을 두 번 계산

2

최적 부분 구조

작은 문제의 답을 모으면 큰 문제의 답이 된다

예) fib(n) = fib(n−1) + fib(n−2)

예제: 피보나치 수열



$F(4) + F(5) = F(6) \rightarrow 3 + 5 = 8$

$$F(n) = F(n-1) + F(n-2)$$

단, $F(0) = 0$, $F(1) = 1$

앞의 두 수를 더하면 다음 수가 됩니다

먼저 재귀로 풀어보기

Python

```python
def fib(n):
    if n <= 1:              # 종료 조건
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(10))   # 55
```

수학 식을 거의 그대로 코드로 옮긴 형태

✓

장점

식이 그대로 보여서 읽기 쉽고 짧다

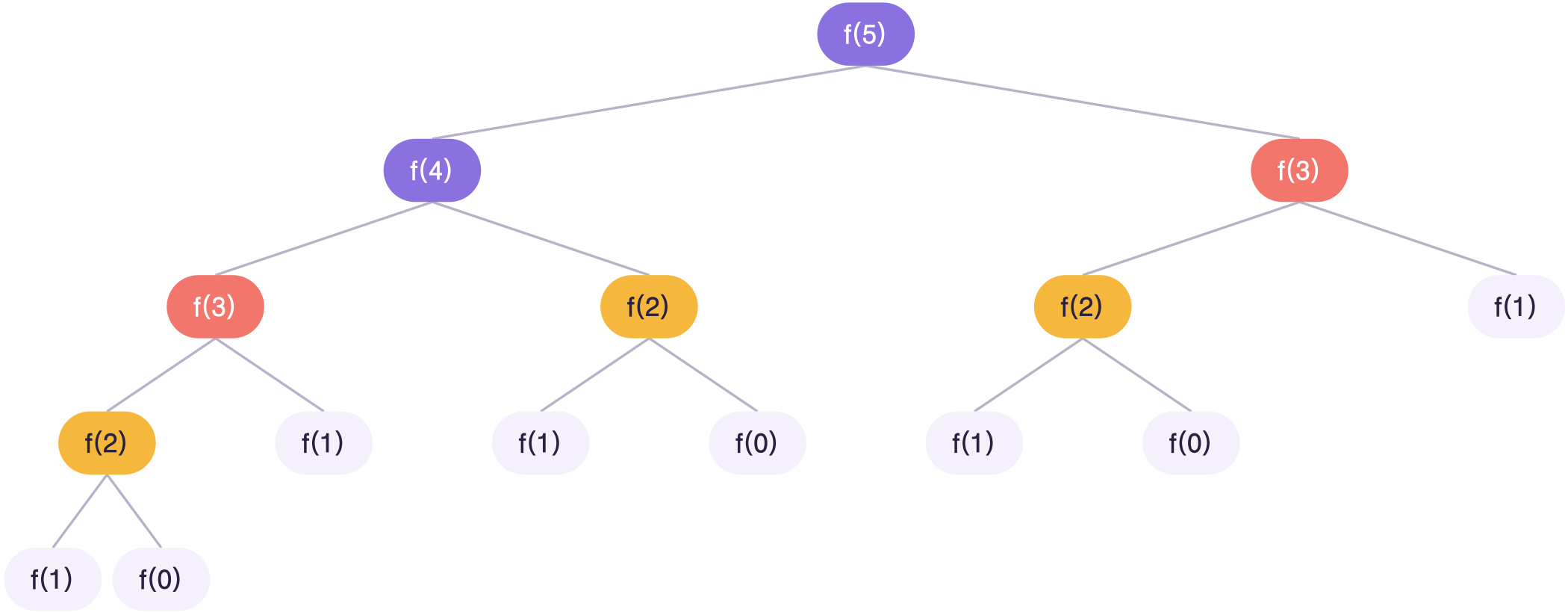
!

문제점

같은 값을 몇 번이고 다시 계산한다
n이 조금만 커져도 엄청 느려짐

왜 느려질까? 호출 과정을 그려보자 →

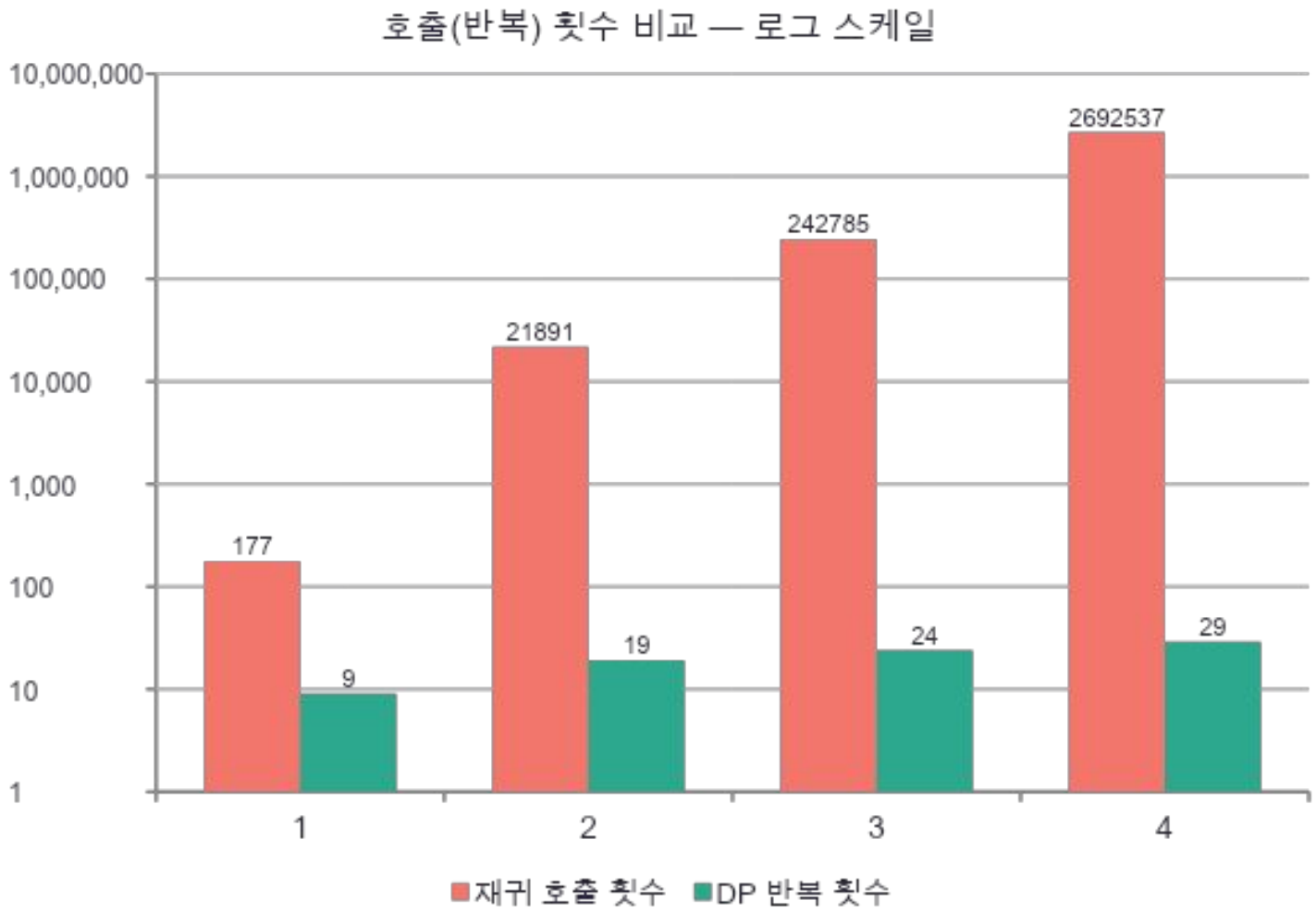
fib(5)의 호출 트리: 같은 계산이 반복된다



f(3) 2번 f(2) 3번 f(1) 5번

총 15번 호출 중 **9번이 이미 했던 계산**

n이 커지면 호출 횟수가 폭발한다



n = 30일 때

2,692,537

재귀 호출 횟수

29

DP 반복 횟수

n이 1 늘 때마다 재귀 호출 수는 약 1.6배씩 늘어나요

해결법 ① 메모이제이션 (Top-down)

Python

```python
memo = {}

def fib(n):
    if n <= 1:
        return n
    if n in memo:          # 이미 계산했다면
        return memo[n]     # 저장된 값 재사용
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

재귀 구조는 그대로, 계산 결과만 딕셔너리에 저장

1

memo부터 확인

계산하기 전에 저장된 값이 있는지 본다

2

있으면 바로 꺼내기

다시 계산하지 않고 그대로 반환

3

없으면 계산 후 저장

다음에 또 필요할 때를 위해 기록

Tip `@lru_cache` 한 줄로도 자동 메모이제이션 (functools)

해결법 ② 타뷸레이션 (Bottom-up)

```python
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)    # 표 만들기
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Python

✓ **재귀 호출이 없다**

for문만 쓰므로
RecursionError 걱정 없음

✓ **작은 것부터 차곡차곡**

dp[0], dp[1]부터 표를 순서대로 채운다



세 가지 방법 한눈에 비교

구분	단순 재귀	메모이제이션 (Top-down)	바텀업 (Bottom-up)
푸는 방향	큰 문제 → 작은 문제	큰 문제 → 작은 문제	작은 문제 → 큰 문제
중복 계산	매우 많음	없음	없음
시간복잡도	$O(2^n)$	O(n)	O(n)
공간복잡도	O(n) 호출 스택	O(n) memo + 스택	O(n) → O(1)까지 가능
재귀 깊이 제한	있음	있음	없음

결론 메모이제이션과 바텀업 모두 O(n)! 아주 큰 n에서는 재귀가 없는 바텀업이 더 안전해요

그래서, 왜 DP가 더 좋을까?

15 → 6

중복 계산 제거

fib(5) 기준, 15번 하던 계산을
서로 다른 6개 값만 한 번씩 계산

O(2ⁿ) → O(n)

압도적인 속도

지수 시간에서 선형 시간으로.
n=30이면 약 269만 번 → 29번

No Error

큰 n도 안전

바텀업은 재귀가 없어서 파이썬
재귀 한도(약 1000)에 걸리지 않음

재귀는 **"또 계산"**, DP는 **"기억해서 재사용"**

DP 문제를 푸는 4단계

1

상태 정의

dp[i]가 무엇을 뜻하는지 정하기

```
dp[i] = i번째
피보나치 수
```

2

점화식 세우기

큰 문제를 작은 문제로 표현하기

```
dp[i] = dp[i-1]
      + dp[i-2]
```

3

초기값 정하기

더 쪼갤 수 없는 가장 작은 답

```
dp[0] = 0
dp[1] = 1
```

4

계산 순서

필요한 값이 먼저 채워지도록

```
for i in
 range(2, n+1)
```

이 4가지만 정하면 대부분의 DP 문제를 같은 틀로 풀 수 있어요

실습: 계단 오르기

문제

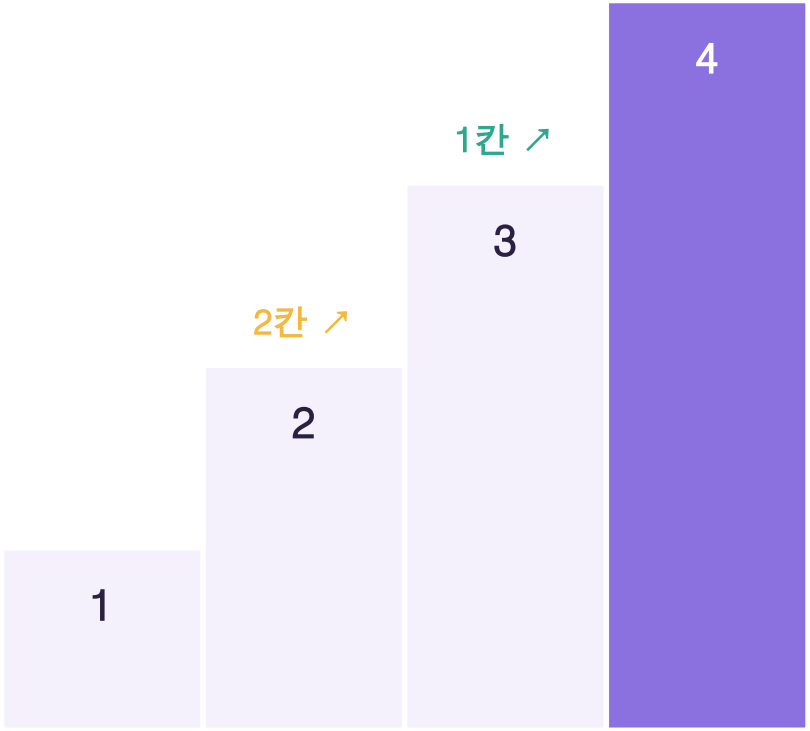
한 번에 1칸 또는 2칸씩 오를 수 있을 때,
n칸짜리 계단을 끝까지 오르는 방법은 몇 가지일까요?

입출력 예시

n = 1	n = 2	n = 3	n = 4
1가지	2가지	3가지	5가지

힌트 n번째 칸에 도착하기 바로 전에는 어디에 있었을까?

4번째 칸에 오는 방법
= 3번째에서 1칸 + 2번째에서 2칸



오늘의 정리

1

DP는 작은 문제의 답을 저장해서 다시 쓰는 방법

중복되는 부분 문제 + 최적 부분 구조가 있을 때 사용

2

피보나치: 재귀 O(2ⁿ) → DP O(n)

같은 값을 한 번만 계산하기 때문

3

DP 구현 방식은 두 가지

메모이제이션(위→아래, 재귀+저장) / 바텀업(아래→위, 반복문+표)